

Tarefa 2: Heap Priority Queue

1. Identificação da Equipe

- Gustavo Bada Zanatta
- Vinícius Moreira Krüger

2. Identificação da Tarefa

Este relatório se refere à Tarefa 2 da disciplina de Estruturas de Dados, cujo objetivo é implementar e analisar uma fila de prioridade baseada em **Heap Priority Queue**.

A atividade solicitou a implementação da estrutura em linguagem C, utilizando como base o arquivo *dataset_v1.csv*, composto por registros com os campos *nome*, *matrícula*, *ranking* e *curso*. Para o experimento de desempenho, a prioridade foi definida pelo campo **matrícula**.

Além da implementação da fila de prioridade, também foi desenvolvida uma aplicação com menu para testar as operações da estrutura e outra aplicação para medir o comportamento da inserção na heap, contando o número médio de iterações realizadas.

3. Explicação conceitual sobre a solução fornecida

3.1 Entrada

A principal fonte de dados do nosso código é o arquivo *dataset_v1.csv*. Ele contém 10 mil linhas, e cada linha representa um registro com *Nome*, *Matrícula*, *Ranking* e *Curso*. Lemos esse arquivo no nosso código em C e carregamos os dados para dentro de um vetor de structs. Para o nosso experimento de desempenho, definimos que o campo **Matrícula** seria o valor de prioridade para a ordenação da Heap.

3.2 Processo

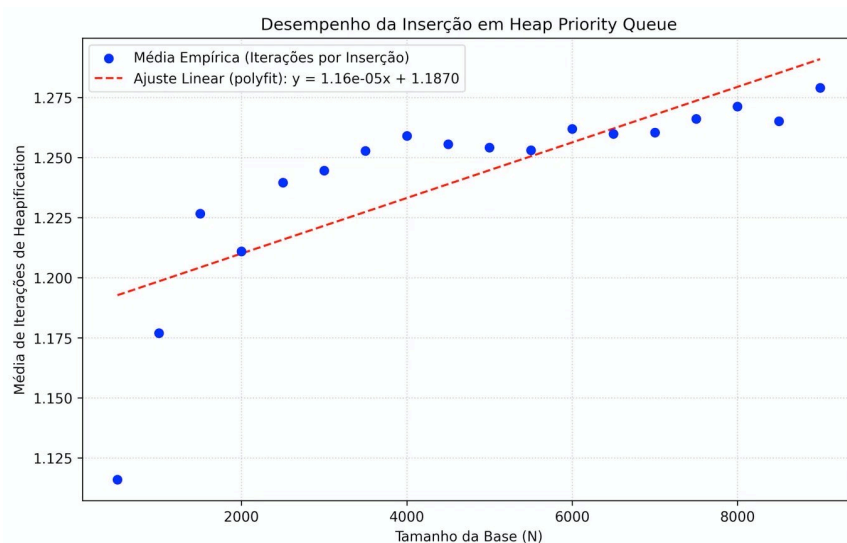
Dividimos o desenvolvimento em etapas. Primeiro, implementamos a Heap mantendo a mesma interface de funções da FDE antiga para facilitar a compatibilidade. Criamos um menu de testes básico, onde o código insere os dados na Heap em lotes crescentes (500, 1000, 1500... até 9000). Para medir o custo, colocamos um contador dentro do laço de inserção (*heapification*). A cada lote processado, pegamos o total de iterações que o laço rodou e dividimos pela quantidade de elementos inseridos, obtendo a média de custo de cada inserção.

3.3 Saída

O programa em C exporta um arquivo de texto com as médias de iterações de cada lote. Pegamos esses pontos gerados (o conjunto R) e usamos um script em Python com a função *numpy.polyfit* para desenhar a curva de tendência e gerar o gráfico do experimento.

Tamanho da base (N)	Média de iterações
500	1.116000
1.000	1.177000
1.500	1.226667
2.000	1.211000
2.500	1.239600
3.000	1.244667
3.500	1.252857
4.000	1.259000
4.500	1.255556
5.000	1.254200
5.500	1.253091
6.000	1.262000
6.500	1.259846
7.000	1.260429
7.500	1.266133
8.000	1.271250
8.500	1.265176
9.000	1.279000

4. Resultados e Análise



Analisando o gráfico e a tabela acima, percebemos que o comportamento do nosso código confirmou exatamente o que estudamos na teoria sobre a *Heap Priority Queue*.

A literatura indica que a inserção na Heap tem um custo de $O(\log N)$ no pior dos casos. Observando os nossos resultados, o número médio de iterações de *heapification* por elemento ficou extremamente baixo e estável, oscilando na faixa de apenas 1 a 1.3 iterações por inserção. Ou seja, mesmo quando o lote já estava em 9.000 registros inseridos, a nova matrícula não precisava percorrer um caminho longo (subindo vários níveis da árvore) para encontrar sua posição correta, observa-se uma leve tendência de alta conforme N aumenta, pois a árvore fica um pouco mais profunda, mas o crescimento é desprezível e tende a um teto constante.

Isso acontece porque as matrículas no arquivo CSV chegam de forma bem "embaralhada" (aleatória). Na média, a maioria dos nós encontra sua posição ideal fazendo apenas 1 ou 2 trocas com os "pais", sem precisar subir até a raiz da árvore.

Concluimos que a *Heap Priority Queue* é absurdamente mais eficiente que as listas encadeadas (como a FDE) para gerenciar prioridades. Se usássemos uma LDE, o custo para achar a posição de inserção seria linear $O(N)$, forçando o programa a percorrer milhares de nós a cada nova matrícula. A Heap resolve o problema saltando níveis, o que justifica totalmente o seu uso em sistemas que exigem alta performance.

5. Conclusão

Em resumo, a realização deste trabalho cumpriu com sucesso o objetivo de tirar a teoria da *Heap Priority Queue* do papel e testá-la em um cenário real. Conseguimos implementar a estrutura em C e provar matematicamente, através do script em Python, que ela é extremamente rápida e eficiente para organizar prioridades.

